Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

Department of Information Technology (B.Sc.I.T Semester V)

Applied Business Analytics

Practical – V

Roll No.: T038	Name: Akansha Pritam Ranade
Class: TYIT	Batch: 1
Date of Assignment: 21-08-2026	Date/Time of Submission: 21-08-2026

5. Probability and Probability Distributions:

a. Simulate a probability experiment (e.g., rolling dice) using programming and calculate the probabilities of different outcomes.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

1. Simulate 1,000 dice rolls and display the first 20 outcomes.

```
np.random.seed(42)
rolls = np.random.randint(1, 6, size=1000)
print(" First 20 outcomes:", rolls[:20])
```

```
First 20 outcomes: [4 5 3 5 5 2 3 3 3 5 4 3 5 2 4 2 4 5 1 4]
```

2. Calculate the experimental probability of getting each number from 1 to 6.

```
counts = pd.Series(rolls).value_counts().sort_index()
exp_prob = counts / 1000
print("\nExperimental Probability:\n", exp_prob)
```

```
 1    0.210
2    0.190
3    0.190
4    0.206
5    0.204
Name: count, dtype: float64
```

3. Calculate the theoretical probability of getting each number from 1 to 6.

4. Find the probability of getting 1, 2, 3, 4, 5, 6.

```
theo_prob = pd.Series(1 / 6, index=range(1, 7))
print("\nTheoretical Probability (for each 1-6):", round(1 / 6, 4))
```

```
3 & 4. Theoretical Probability (for each 1-6): 0.1667
```

5. Find the probability of getting an even number.

6. Find the probability of getting an odd number.

7. Find the probability of getting a number greater than 4.

8. Find the probability of getting a number less than 4.

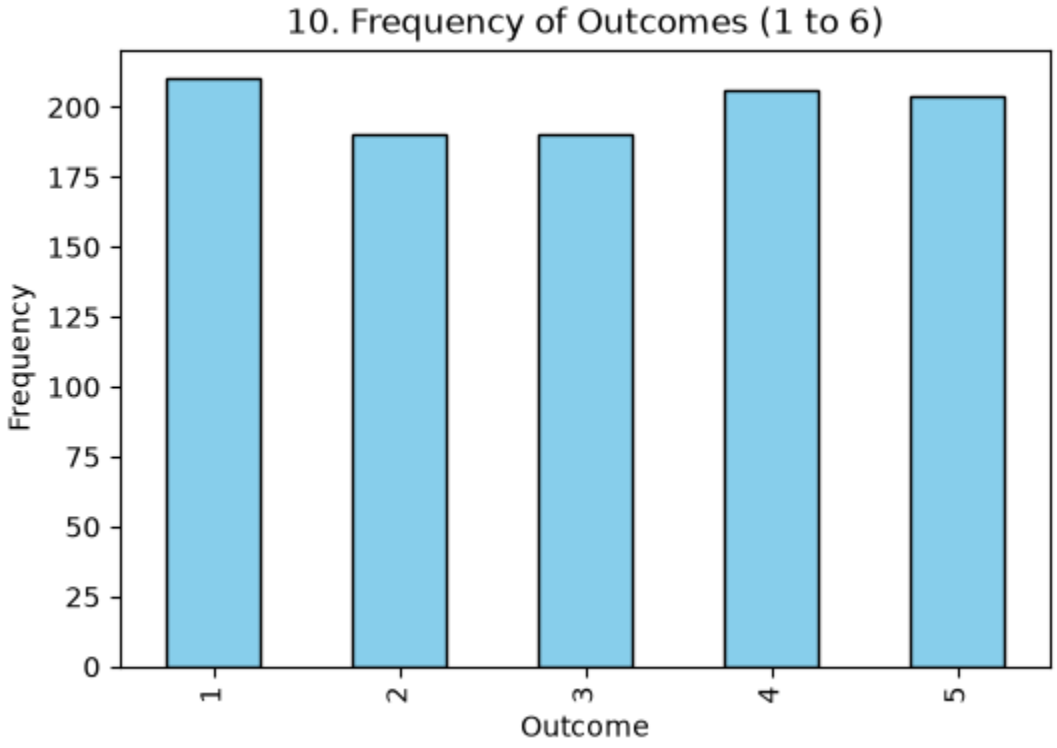
9. Find the probability of getting a number greater than or equal to 4.

```
print("\n5. P(Even):", np.mean(rolls % 2 == 0))
print("6. P(Odd):", np.mean(rolls % 2 != 0))
print("7. P(Number > 4):", np.mean(rolls > 4))
print("8. P(Number < 4):", np.mean(rolls < 4))
print("9. P(Number >= 4):", np.mean(rolls >= 4))
```

```
5. P(Even): 0.396
6. P(Odd): 0.604
7. P(Number > 4): 0.204
8. P(Number < 4): 0.59
9. P(Number >= 4): 0.41
```

10. Create a bar chart showing the frequency of outcomes 1 to 6.

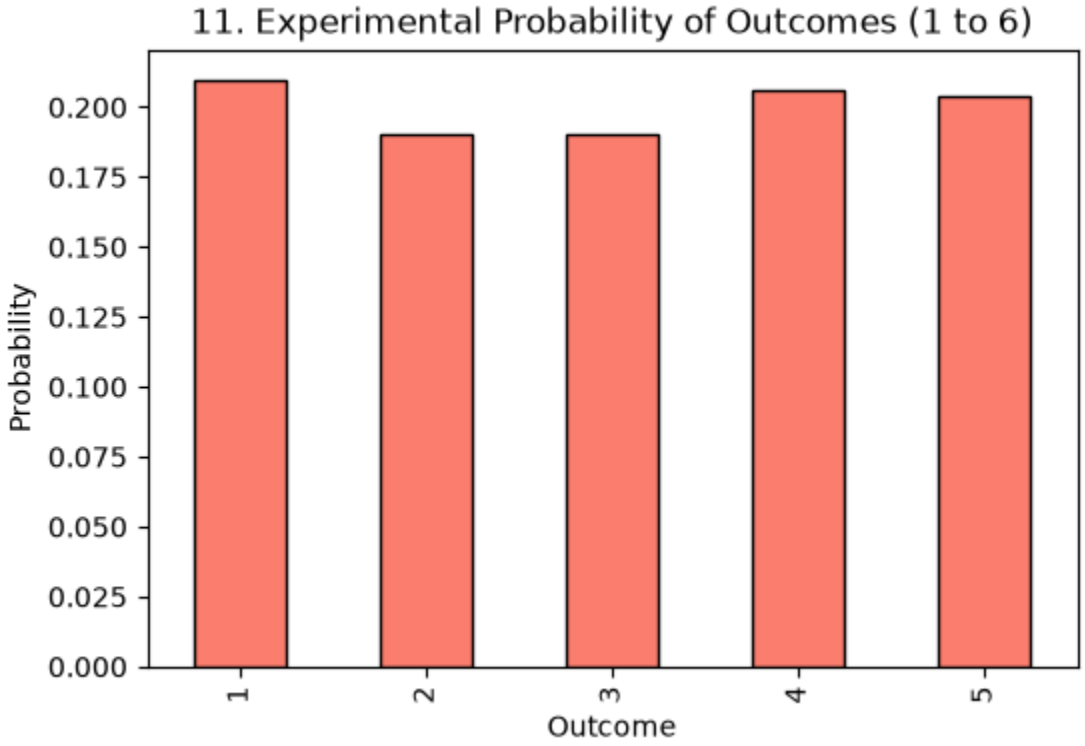
```
plt.figure(figsize=(6, 4))
counts.plot(kind="bar", color="skyblue", edgecolor="black")
plt.title("10. Frequency of Outcomes (1 to 6)")
plt.xlabel("Outcome")
plt.ylabel("Frequency")
plt.show()
```



11. Create a bar chart showing the experimental probability of outcomes 1 to 6.

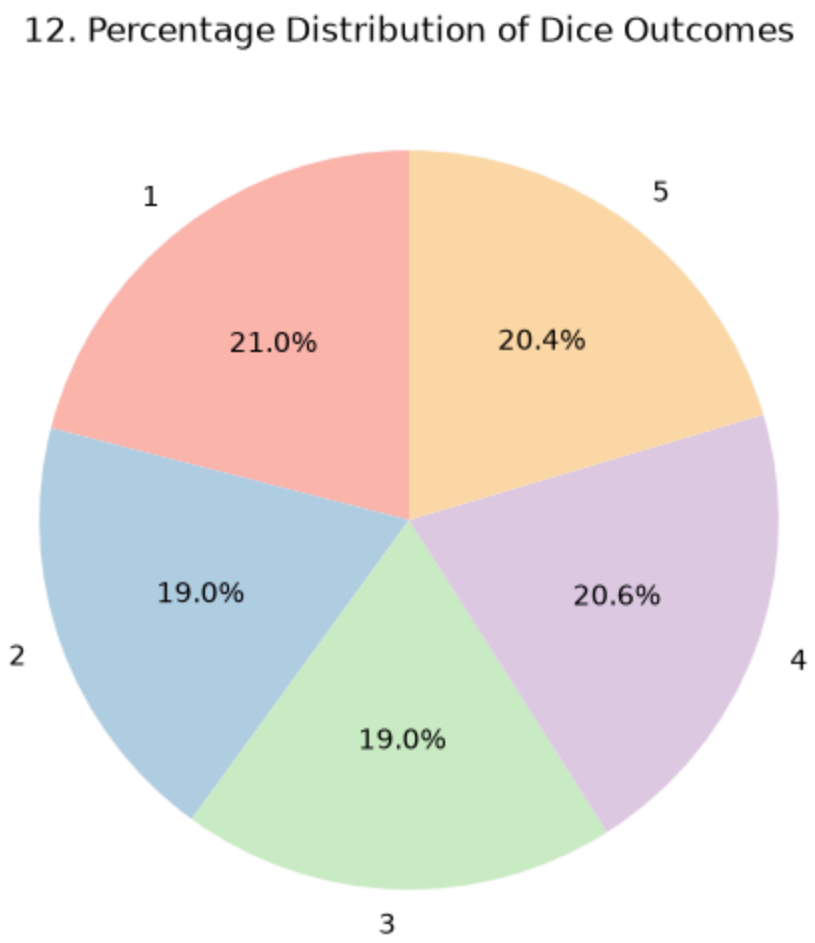
```
plt.figure(figsize=(6, 4))
exp_prob.plot(kind="bar", color="salmon", edgecolor="black")
plt.title("11. Experimental Probability of Outcomes (1 to 6)")
plt.xlabel("Outcome")
```

```
plt.ylabel("Probability")
plt.show()
```



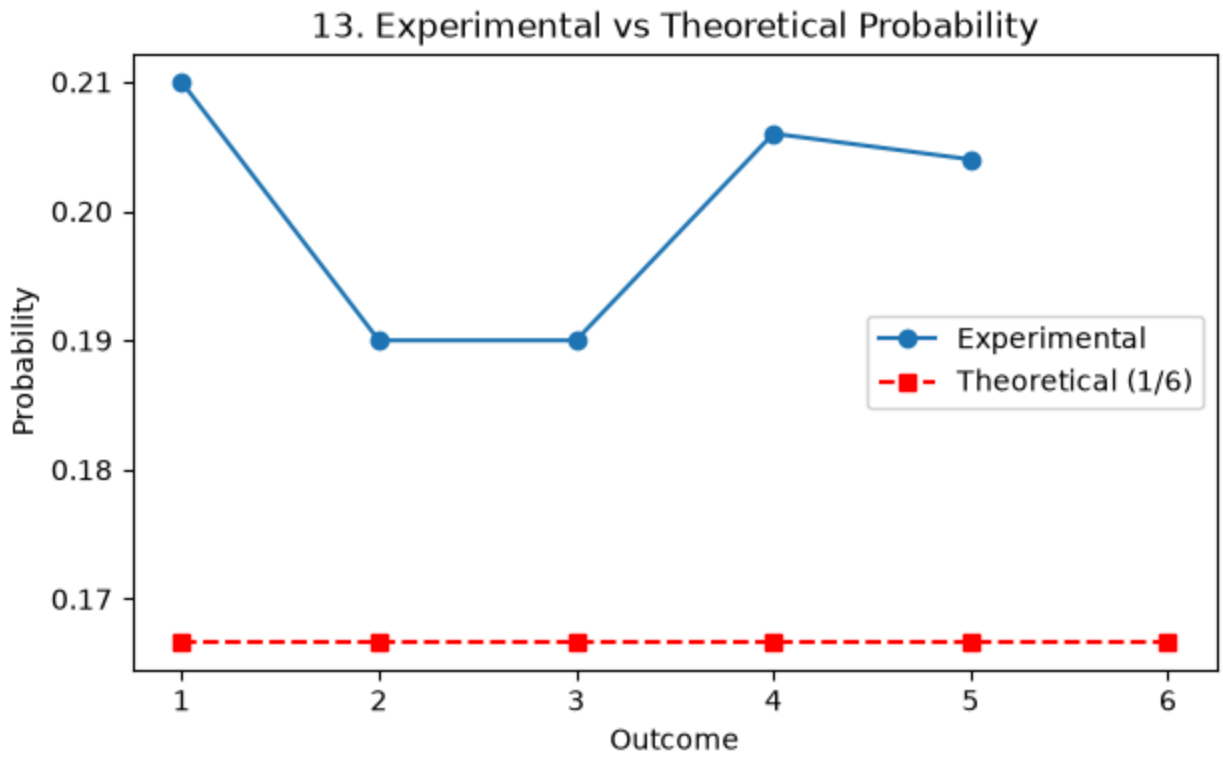
12. Create a pie chart showing the percentage distribution of dice outcomes.

```
plt.figure(figsize=(6, 6))
plt.pie(
    counts,
    labels=counts.index,
    autopct="%1.1f%%",
    startangle=90,
    colors=plt.cm.Pastel1.colors,
)
plt.title("12. Percentage Distribution of Dice Outcomes")
plt.show()
```



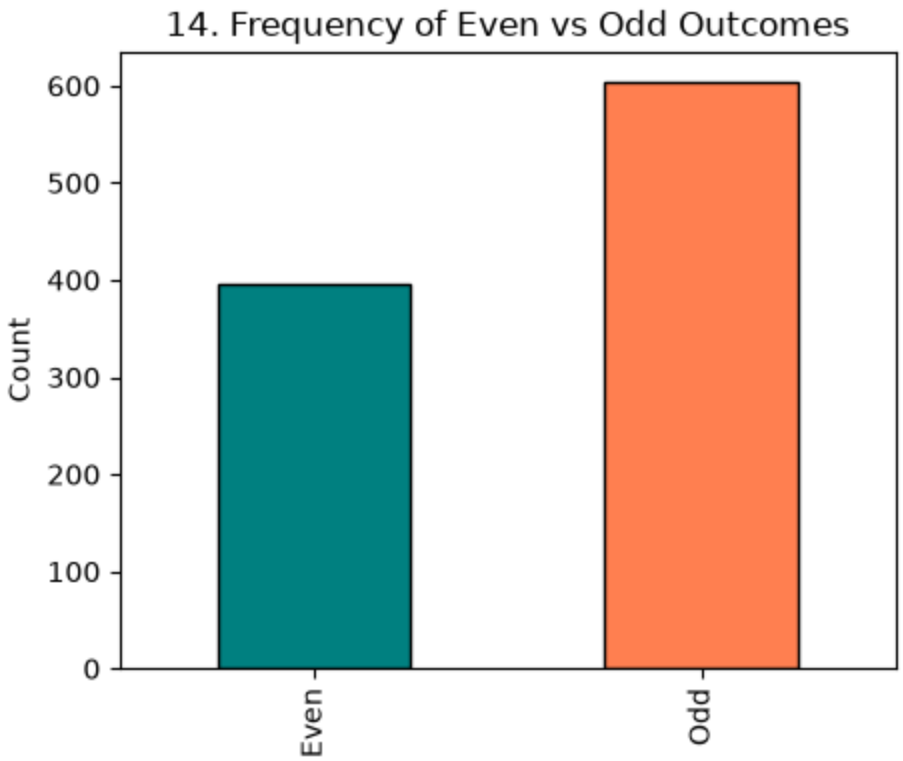
13. Create a line chart comparing experimental probability and theoretical probability for outcomes 1 to 6.

```
plt.figure(figsize=(7, 4))
plt.plot(exp_prob.index, exp_prob.values, marker="o", label="Experimental")
plt.plot(
    theo_prob.index,
    theo_prob.values,
    marker="s",
    linestyle="--",
    color="red",
    label="Theoretical (1/6)",
)
plt.title("13. Experimental vs Theoretical Probability")
plt.xlabel("Outcome")
plt.ylabel("Probability")
plt.legend()
plt.show()
```



14. Create a chart showing the frequency of Even vs Odd outcomes.

```
even_odd_counts = pd.Series(
    {"Even": np.sum(rolls % 2 == 0), "Odd": np.sum(rolls % 2 != 0)}
)
plt.figure(figsize=(5, 4))
even_odd_counts.plot(kind="bar", color=["teal", "coral"], edgecolor="black")
plt.title("14. Frequency of Even vs Odd Outcomes")
plt.ylabel("Count")
plt.show()
```



15.Create a chart showing the experimental probability of:

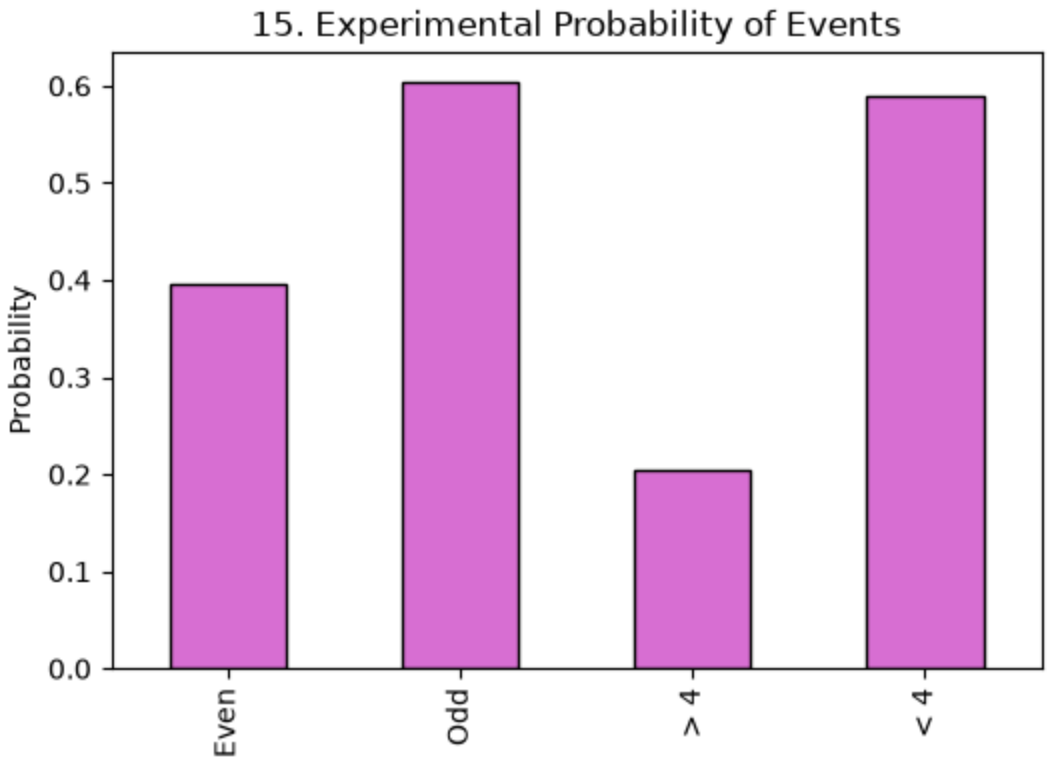
• Even numbers

• Odd numbers

• Numbers greater than 4

• Numbers less than 4

```
event_probs = pd.Series({
    "Even": np.mean(rolls % 2 == 0),
    "Odd": np.mean(rolls % 2 != 0),
    "> 4": np.mean(rolls > 4),
    "< 4": np.mean(rolls < 4),
})
plt.figure(figsize=(6, 4))
event_probs.plot(kind="bar", color="orchid", edgecolor="black")
plt.title("15. Experimental Probability of Events")
plt.ylabel("Probability")
plt.show()
```



b. Generate random numbers from various probability distributions (normal, uniform, exponential) and analyze their properties.

```
import matplotlib.pyplot as plt
import numpy as np
from scipy import stats
```

1. Generate 1,000 random numbers from a normal distribution with:
Mean = 50 and Standard deviation = 10

```
np.random.seed(42)
norm_data = np.random.normal(loc=50, scale=10, size=1000)
```

2. Calculate the mean, median, mode, variance, and standard deviation of the generated data.
3. Calculate the following statistics for the generated Normal distribution:

- **Mean**
- **Median**
- **Variance**
- **Standard deviation**
- **Minimum**
- **Maximum**

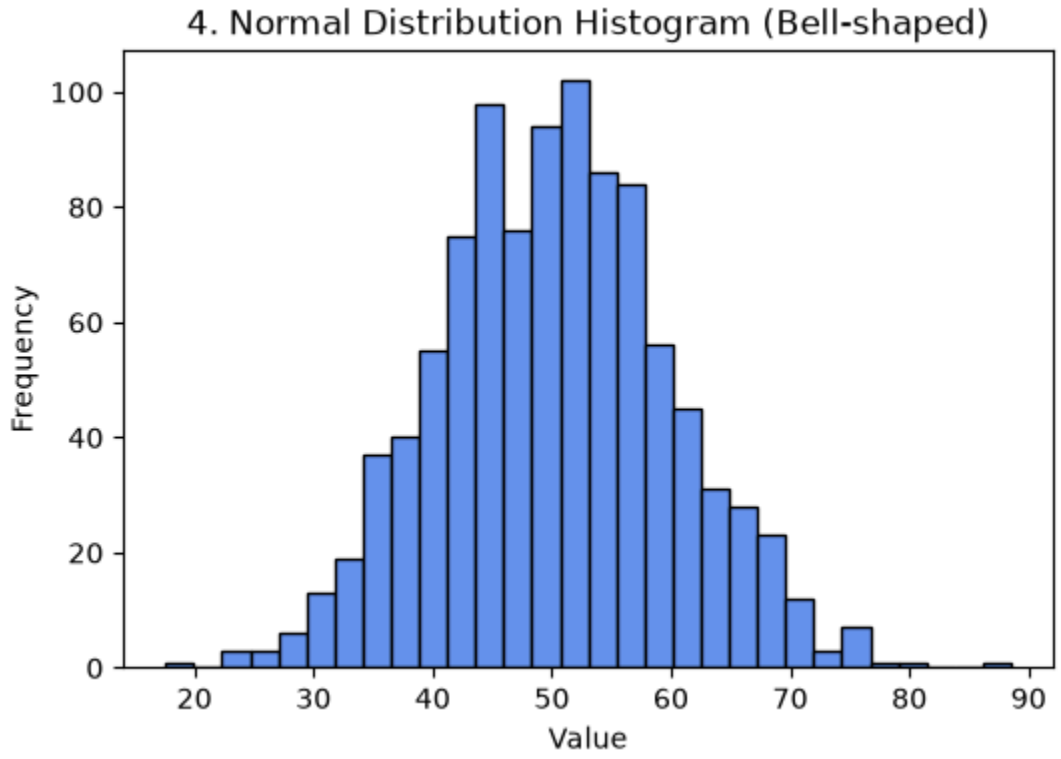
```
print("=== Normal Distribution Statistics ===")
print("Mean:", np.mean(norm_data))
print("Median:", np.median(norm_data))
print("Mode (binned/rounded):", stats.mode(np.round(norm_data))[0])
print("Variance:", np.var(norm_data, ddof=1))
print("Standard Deviation:", np.std(norm_data, ddof=1))
```

```
print("Minimum:", np.min(norm_data))
print("Maximum:", np.max(norm_data))
```

```
=== Normal Distribution Statistics ===
Mean: 50.193320558223256
Median: 50.25300612234888
Mode (binned/rounded): 52.0
Variance: 95.88638535851024
Standard Deviation: 9.792159381796756
Minimum: 17.58732659930927
Maximum: 88.52731490654722
```

4. Create a histogram of the Normal distribution and analyze its shape.

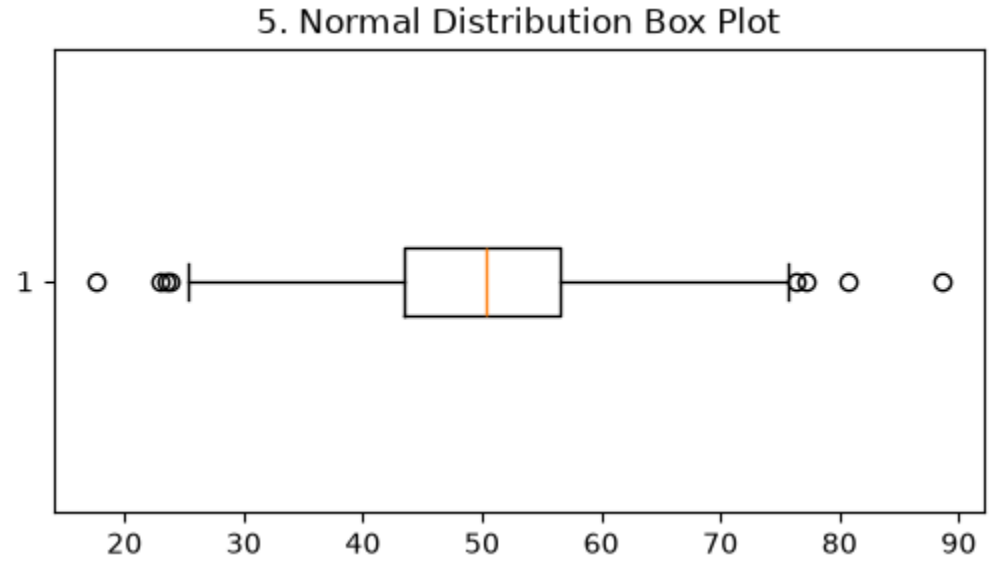
```
plt.figure(figsize=(6, 4))
plt.hist(norm_data, bins=30, color="cornflowerblue", edgecolor="black")
plt.title("4. Normal Distribution Histogram (Bell-shaped)")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```



5. Create a box plot of the Normal distribution.

```
plt.figure(figsize=(6, 3))
plt.boxplot(norm_data, vert=False)
plt.title("5. Normal Distribution Box Plot")
plt.show()
```

```
plt.boxplot(norm_data, vert=False)
```



6. Calculate the percentage of values that fall between: 40<=X<=60
7. Calculate the percentage of values that fall between: 30<=X<=70

```
pct_40_60 = np.mean((norm_data >= 40) & (norm_data <= 60)) * 100
pct_30_70 = np.mean((norm_data >= 30) & (norm_data <= 70)) * 100
print(f"6. Percentage between 40 <= X <= 60: {pct_40_60:.2f}% (approx 68%)")
print(f"7. Percentage between 30 <= X <= 70: {pct_30_70:.2f}% (approx 95%)")
```

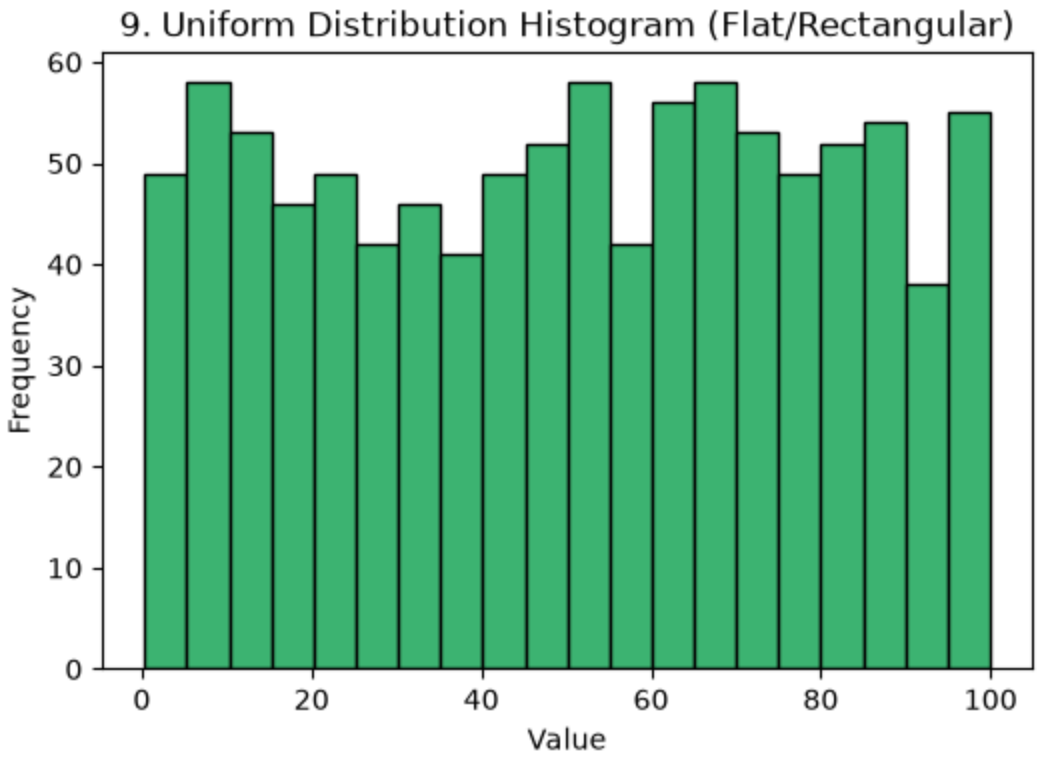
```
6. Percentage between 40 <= X <= 60: 69.80% (approx 68%)
7. Percentage between 30 <= X <= 70: 95.90% (approx 95%)
```

8. Generate 1,000 random numbers from a uniform distribution between 0 and 100.

```
uni_data = np.random.uniform(low=0, high=100, size=1000)
```

9. Create a histogram of the Uniform distribution.

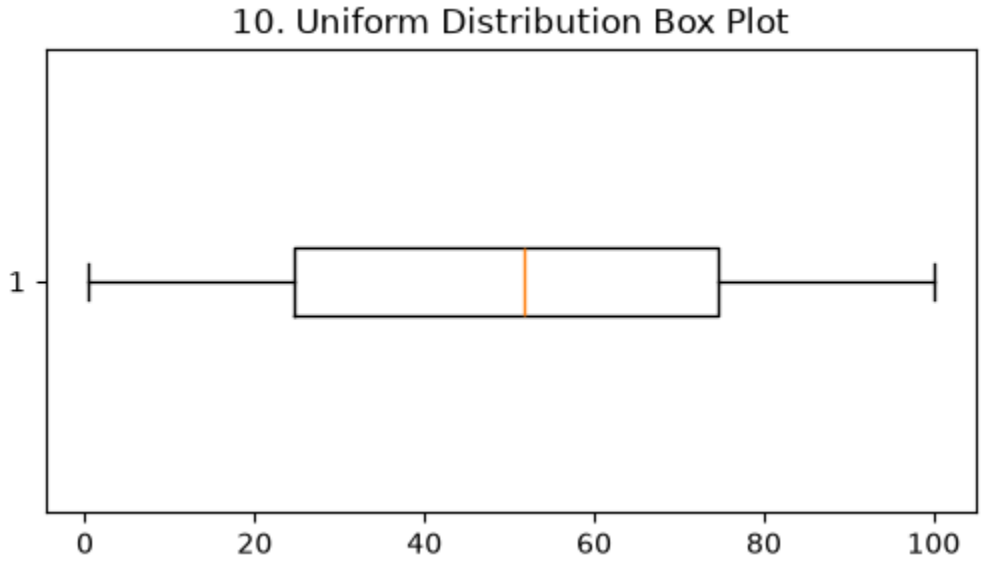
```
plt.figure(figsize=(6, 4))
plt.hist(uni_data, bins=20, color="mediumseagreen", edgecolor="black")
plt.title("9. Uniform Distribution Histogram (Flat/Rectangular)")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```



10. Create a box plot of the Uniform distribution.

```
plt.figure(figsize=(6, 3))
plt.boxplot(uni_data, vert=False)
plt.title("10. Uniform Distribution Box Plot")
plt.show()
```

```
plt.boxplot(uni_data, vert=False)
```

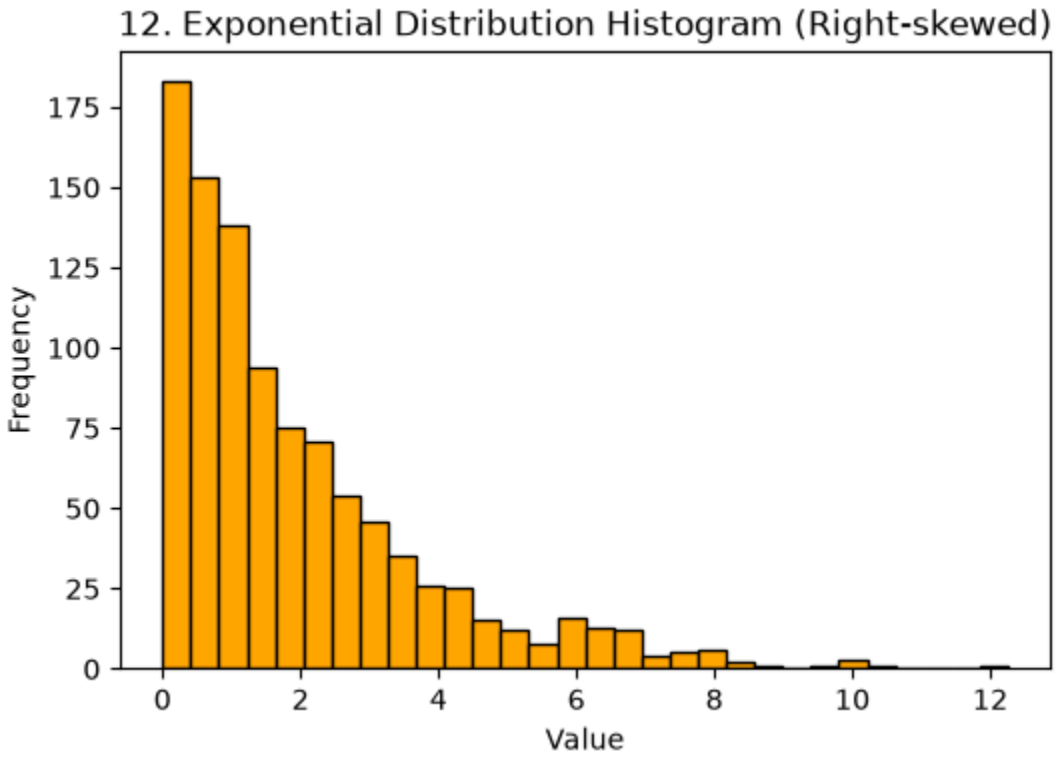


11. Generate 1,000 random numbers from an Exponential distribution with scale = 2.

exp_data = np.random.exponential(scale=2, size=1000)

12. Create a histogram of the Exponential distribution.

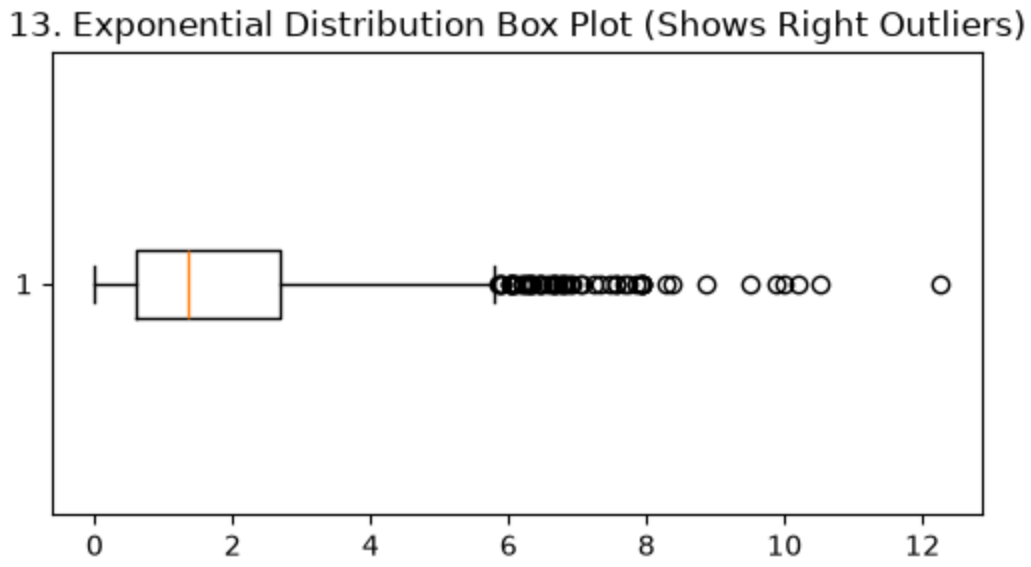
```
plt.figure(figsize=(6, 4))
plt.hist(exp_data, bins=30, color="orange", edgecolor="black")
plt.title("12. Exponential Distribution Histogram (Right-skewed)")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```



13. Create a box plot and identify possible extreme values.

```
plt.figure(figsize=(6, 3))
plt.boxplot(exp_data, vert=False)
plt.title("13. Exponential Distribution Box Plot (Shows Right Outliers)")
plt.show()
```

```
plt.boxplot(exp_data, vert=False)
```



14. Calculate the percentage of values greater than 2.
15. Calculate the percentage of values greater than 5.

```
pct_gt_2 = np.mean(exp_data > 2) * 100
pct_gt_5 = np.mean(exp_data > 5) * 100
print(f"14. Percentage of values > 2: {pct_gt_2:.2f}%")
print(f"15. Percentage of values > 5: {pct_gt_5:.2f}%")
```

```
14. Percentage of values > 2: 36.50%
15. Percentage of values > 5: 8.20%
```

C) Coin toss task (Include Prints)

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
np.random.seed(42)
```

1. Simulate 1,000 coin tosses and calculate the experimental probability of:
• Getting Heads
• Getting Tails

```
tosses_1k = np.random.choice(["Heads", "Tails"], size=1000)
heads_1k = np.sum(tosses_1k == "Heads")
tails_1k = np.sum(tosses_1k == "Tails")
```

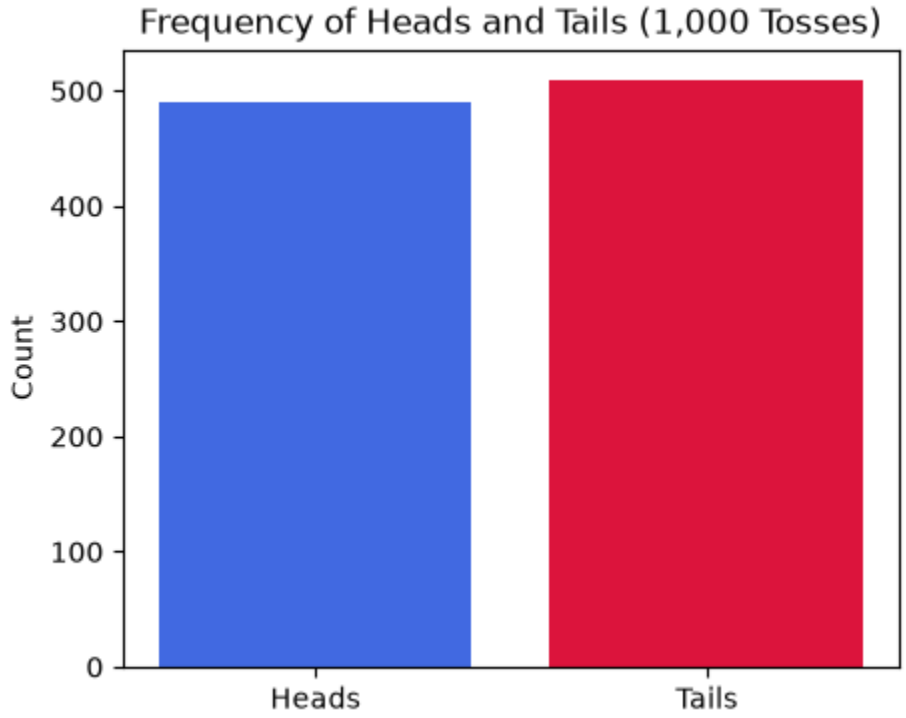
2. Calculate the theoretical probability of Heads and Tails.

```
print("=== 1,000 Coin Toss Experiment ===")
print(f"Experimental Probability (Heads): {heads_1k / 1000:.4f}")
print(f"Experimental Probability (Tails): {tails_1k / 1000:.4f}")
print("Theoretical Probability (Heads & Tails): 0.5000\n")
```

```
=== 1,000 Coin Toss Experiment ===
Experimental Probability (Heads): 0.4900
Experimental Probability (Tails): 0.5100
Theoretical Probability (Heads & Tails): 0.5000
```

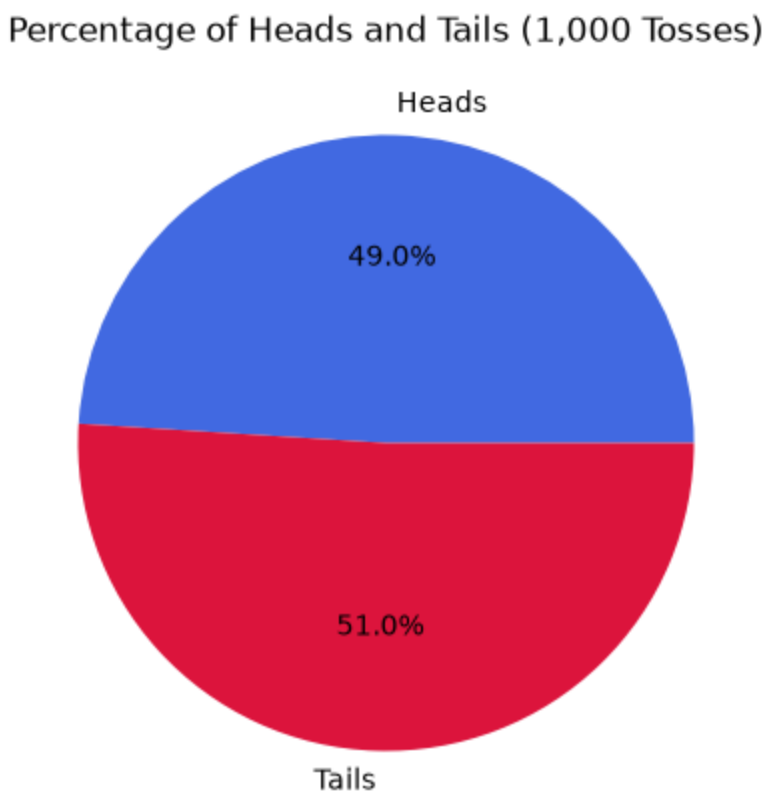
3. Create a bar chart showing the frequency of Heads and Tails.

```
plt.figure(figsize=(5, 4))
plt.bar(["Heads", "Tails"], [heads_1k, tails_1k], color=["royalblue", "crimson"])
plt.title("Frequency of Heads and Tails (1,000 Tosses)")
plt.ylabel("Count")
plt.show()
```



4. Create a pie chart showing the percentage of Heads and Tails.

```
plt.figure(figsize=(5, 5))
plt.pie(
    [heads_1k, tails_1k],
    labels=["Heads", "Tails"],
    autopct="%1.1f%%",
    colors=["royalblue", "crimson"],
)
plt.title("Percentage of Heads and Tails (1,000 Tosses)")
plt.show()
```



5. If a coin is tossed 10 times and Heads occurs 8 times, calculate the experimental probability of Heads.

print("5. 10 tosses with 8 Heads -> Experimental Probability:", 8 / 10)

```
5. 10 tosses with 8 Heads -> Experimental Probability: 0.8
```

6. Simulate 10,000 coin tosses and create a complete analysis containing:

- **Total tosses**
- **Number of Heads**
- **Number of Tails**
- **Experimental probability of Heads**
- **Experimental probability of Tails**
- **Theoretical probabilities**
- **Difference between experimental and theoretical probabilities**
- **Bar chart**
- **Pie chart**

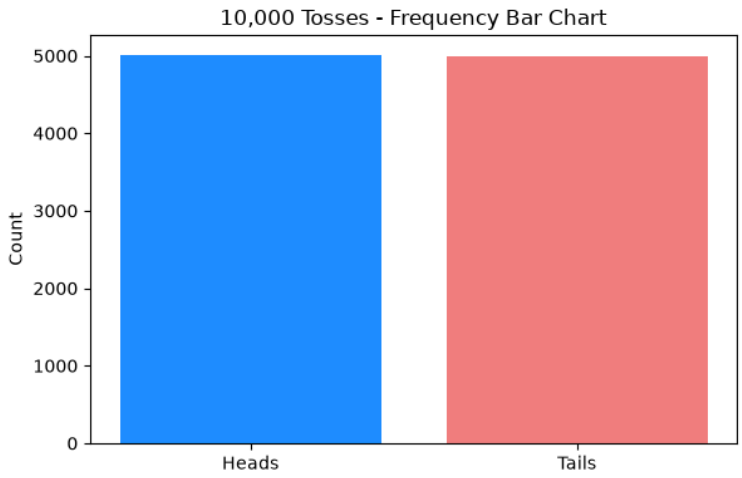
```
n = 10000
tosses_10k = np.random.choice(["Heads", "Tails"], size=n)
heads_10k = np.sum(tosses_10k == "Heads")
tails_10k = np.sum(tosses_10k == "Tails")
exp_p_h = heads_10k / n
exp_p_t = tails_10k / n
theo_p = 0.5
```

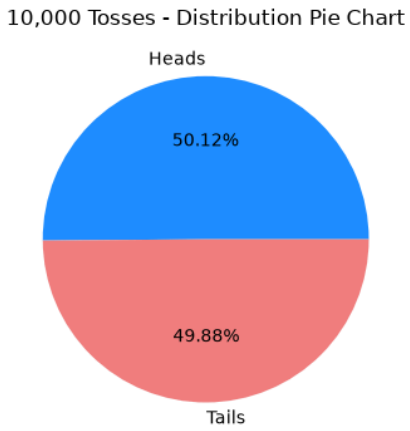
```
diff_h = abs(exp_p_h - theo_p)
diff_t = abs(exp_p_t - theo_p)
print("\n=== 10,000 Coin Tosses Complete Report ===")
print("• Total tosses:", n)
print("• Number of Heads:", heads_10k)
print("• Number of Tails:", tails_10k)
print(f"• Experimental probability of Heads: {exp_p_h:.4f}")
print(f"• Experimental probability of Tails: {exp_p_t:.4f}")
print(f"• Theoretical probabilities: {theo_p:.4f}")
print(
    f"• Difference between experimental and theoretical (Heads): {diff_h:.4f}"
)
print(
    f"• Difference between experimental and theoretical (Tails): {diff_t:.4f}"
)
```

```
=== 10,000 Coin Tosses Complete Report ===
• Total tosses: 10000
• Number of Heads: 5012
• Number of Tails: 4988
• Experimental probability of Heads: 0.5012
• Experimental probability of Tails: 0.4988
• Theoretical probabilities: 0.5000
• Difference between experimental and theoretical (Heads): 0.0012
• Difference between experimental and theoretical (Tails): 0.0012
```

```
# Charts for 10,000 Tosses
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].bar(
    ["Heads", "Tails"],
    [heads_10k, tails_10k],
    color=["dodgerblue", "lightcoral"],
)
ax[0].set_title("10,000 Tosses - Frequency Bar Chart")
ax[0].set_ylabel("Count")

ax[1].pie(
    [heads_10k, tails_10k],
    labels=["Heads", "Tails"],
    autopct="%1.2f%%",
    colors=["dodgerblue", "lightcoral"],
)
ax[1].set_title("10,000 Tosses - Distribution Pie Chart")
plt.tight_layout()
plt.show()
```


10,000 Tosses - Frequency Bar Chart
Count
5000
4000
3000
2000
1000
0
Heads
Tails


10,000 Tosses - Distribution Pie Chart
Heads
50.12%
49.88%
Tails